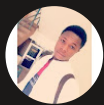




nextwork.org

Cloud Security with AWS IAM



Ahmed Tetteh

Modify permissions in NextWorkDevEnvironmentPolicy [info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": "ec2:*",
7        "Resource": "*",
8        "Condition": {
9          "StringEquals": {
10             "ec2:ResourceTag/Env": "development"
11           }
12         }
13       },
14       {
15         "Effect": "Allow",
16         "Action": "ec2:Describe*",
17         "Resource": "*"
18       },
19       {
20         "Effect": "Deny",
21         "Action": [
22           "ec2:DeleteTags",
23           "ec2:CreateTags"
24         ],
25         "Resource": "*"
26       }
27     ]
28   }
```

[+ Add new statement](#)

JSON Ln 28, Col 1

Security: 0 Errors: 0 Warnings: 0 Suggestions: 0

Edit statement

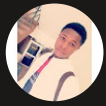
Select a statement

Select an existing statement in the policy or add a new statement.

[+ Add new statement](#)

5851 of 6144 characters remaining

[Check for new access](#)



Introducing Today's Project!

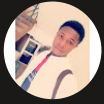
In this project, I will demonstrate how to create IAM Users and policies to authenticate and authorize access to AWS resources like Amazon EC2. I'm doing this project to learn how to launch an EC2 instance and create some few policies and user groups for it.

Tools and concepts

Services I used were Amazon EC2, IAM (Users, Groups, Account Alias, Policy Generators, and Policy Simulators). Key concepts I learnt included how to understand and generate policies using JSON and the Policy Generator, attaching policies to IAM Users to restrict access to certain actions within the AWS account, and lastly, how to use the Policy Simulator to validate a policy before attaching it to an AWS resource or a user.

Project reflection

This project took me approximately 2 hrs, including background studies. The most challenging part was understanding this JSON policy as it contained multiple statements, and how to write and generate the right policies using JSON. It was most rewarding to see that my policy was working denying any actionable rights to the IAM user for the production instance, while keeping all actions restricted to the development instance.

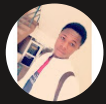


Tags

Tags are like labels or extra names we give to resources in our AWS environment for purposes of filtering, policy application and cost allocation.

The tag I've used on my EC2 instances is called Environment. The value I've assigned for my instances are production and development.

Tags	
Key	Value
Name	nextwork-prod-fap
Env	production



IAM Policies

IAM Policies are rules or permission granted to a user or a resource and defines what they are allowed to do.

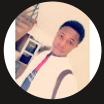
The policy I set up

For this project, I've set up a policy using JSON file to grant all actionable rights to the development instance, read permissions to all EC2 resources, while strictly denying all rights to delete or create tags for EC2 resources.

I've created a policy that allows comprehensive management of EC2 instances tagged as Env:development, permits viewing details of all EC2 resources, but strictly prohibits creating or deleting tags on any EC2 resource.

When creating a JSON policy, you have to define its Effect, Action and Resource.

The Effect, Action, and Resource attributes of a JSON policy means to allow or deny an action (Effect), while Action simply refers to the list of actions the policy allows or denies. Lastly, Resource refers to the various resources being applied to.



My JSON Policy

Modify permissions in NextWorkDevEnvironmentPolicy [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "ec2:*",
7       "Resource": "**",
8       "Condition": {
9         "StringEquals": {
10          "ec2:ResourceTag/Env": "development"
11        }
12      }
13    },
14    {
15      "Effect": "Allow",
16      "Action": "ec2:Describe*",
17      "Resource": "**"
18    },
19    {
20      "Effect": "Deny",
21      "Action": [
22        "ec2:DeleteTags",
23        "ec2:CreateTags"
24      ],
25      "Resource": "**"
26    }
27  ]
28 }
```

[+ Add new statement](#)

Visual **JSON** Actions [+](#)

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

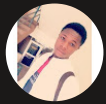
[+ Add new statement](#)

JSON Ln 28, Col 1

5851 of 6144 characters remaining

Security: 0 Errors: 0 Warnings: 0 Suggestions: 0

[Check for new access](#)



Account Alias

An account alias is an alternate name for your AWS account, providing a more user friendly URL sign-in interface for access to the AWS management console.

Creating an account alias took me 1 minute. Now, my new AWS console sign-in URL is <https://nextwork-alias-fap.signin.aws.amazon.com/console>

Create alias for AWS account 007399901516

Preferred alias

nextwork-alias-fap

Must be not more than 63 characters. Valid characters are a-z, 0-9, and - (hyphen).

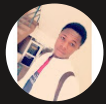
New sign-in URL

<https://nextwork-alias-fap.signin.aws.amazon.com/console>

 IAM users will still be able to use the default URL containing the AWS account ID.

Cancel

Create alias



IAM Users and User Groups

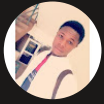
Users

IAM users are people or entities with access to AWS resources within your AWS environment.

User Groups

IAM user groups are a collection of IAM users created to manage all users within the group simultaneously, by attaching policies to the group, rather than individual users.

I attached the policy I created to this user group, which means that all users within this group will be managed under this group policy. They will each have the same rights (do's and don'ts).



Logging in as an IAM User

The first way is to email the user with the sign-in instructions, while the second way is to download the .csv file with the login details for the user.

Once I logged in as my IAM user, I noticed that there were many panels showing Access Denied to read to grab any information from them. This was because of the group policy attached to the user group, in which this IAM user is a part of. The policy will only allow read access to all EC2 instances, actionable rights to the instances with the Tag development, and deny the creation or deletion of all EC2 tags. So, anything outside of this policy by default will be denied, including all information displayed on the dashboard panels and all other AWS services.

Retrieve password

You can view and download the user's password below or email users instructions for signing in to the AWS Management Console. This is the only time you can view and download this password.

Console sign-in details

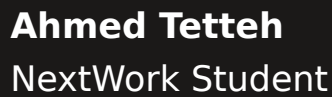
Console sign-in URL
<https://nextwork-alias-fap.signin.aws.amazon.com/console>

User name
[nextwork-dev-luis](#)

Console password
***** [Show](#)

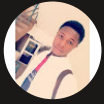
[Email sign-in instructions](#)

[Cancel](#) [Download .csv file](#) [Return to users list](#)



Stopping the production instance

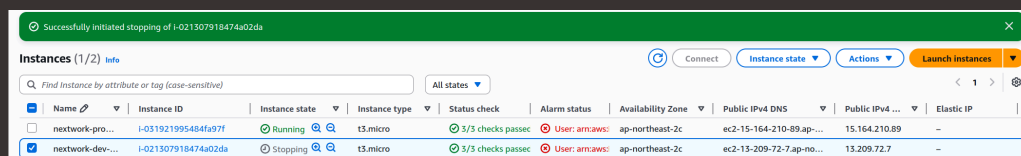
[illegible]



Testing IAM Policies

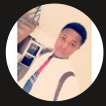
Stopping the development instance

Next, when I tried to stop the development instance, it was successfully initiated. This was because the policy allowed the IAM user to have the rights to stop the development instance.



A screenshot of the AWS Management Console 'Instances' page. A green notification banner at the top states 'Successfully initiated stopping of i-021307918474a02da'. The table below shows two instances. The first instance, 'nextwork-pro...', is in a 'Running' state. The second instance, 'nextwork-dev...', is in a 'Stopping' state, indicated by a blue 'S' icon and a 'Stopping' label. The 'nextwork-dev...' instance is selected with a blue checkmark in the leftmost column.

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
☐	nextwork-pro...	i-031921995484fa97f	Running	t3.micro	3/3 checks passed	User: amzaws	ap-northeast-2c	ec2-15-164-210-89.ap-...	15.164.210.89	-
☒	nextwork-dev...	i-021307918474a02da	Stopping	t3.micro	3/3 checks passed	User: amzaws	ap-northeast-2c	ec2-13-209-72-7.ap-no...	13.209.72.7	-

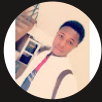


The IAM Policy Simulator

The IAM Policy Simulator can be used to validate your policy, verify if it does what it was intended for and can also be used to check for any errors in the policy. It's also useful for validating the effect of the policy before it is attached to a resource without affecting the actual AWS resource.

How I used the simulator

I set up a simulation for both Delete Tags and Stop Instances for the EC2 instances. The results were positive, as the policy behaved as expected by denying the ability to Delete Tags or Stop Instances for both EC2 instances. I had to adjust the Stop Instance policy menu to simulate for only the development tagged instance as the Deny rule always supersedes the Allow effect. By default, the simulator was referring to all the EC2 instances, hence, the deny rule was in effect. So, by pointing the simulator to the development instance, it displayed the Allow effect, as was originally written.



Policy Simulator

Amazon EC22 Action(s) sele...Select AllDeselect AllReset ContextsClear ResultsRun Simulation

Global Settings

Action Settings and Results [2 actions selected. 0 actions not simulated. 1 actions allowed. 1 actions denied.]

Service	Action	Resource Type	Simulation Resource	Permission
Amazon EC2	DeleteTags	not required	*	denied 1 matching statements.
Amazon EC2	StopInstances	instance	*	allowed 1 matching statements.

[Show statement in NextWorkDevEnvironmentPolicy \(IAM Policy\)](#)

Resource You can specify the resource and context keys used to simulate this action. By default the simulation resource is "".

instance

*

ec2:resourcetag/envdevelopment



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

